

Model Evaluation

0.1 Objectives

- Use training, testing and cross validation to improve the performance of the dataset.
- Identify the point of overfitting of a model
- Use Ridge Regression to identify the change in performance of a model based on its hyperparameters
- Use Grid Search to identify the best performing model using different hyperparameters

Importing Required Libraries

```
[1]: from tqdm import tqdm
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
from sklearn.model_selection import train_test_split, cross_val_score, \
    GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import PolynomialFeatures
```

```
[2]: filepath = "/home/djodj/Downloads/laptop_pricing_dataset.csv"
```

```
[3]: df = pd.read_csv(filepath, header=0)
```

Print the value

```
[4]: df.head()
```

```
[4]:   Unnamed: 0.1  Unnamed: 0  Manufacturer  Category  GPU  OS  CPU_core  \
0              0            0         Acer         4    2    1           5
1              1            1          Dell         3    1    1           3
2              2            2          Dell         3    1    1           7
3              3            3          Dell         4    2    1           5
```

2

	4	4	4	HP	4	2	1	7
	Screen_Size_inch	CPU_frequency	RAM_GB		Storage_GB_SSD		Weight_pounds	
0	14.0	0.551724	8		256		3.52800	
1	15.6	0.689655	4		256		4.85100	
2	15.6	0.931034	8		256		4.85100	
3	13.3	0.551724	8		128		2.69010	
4	15.6	0.620690	8		256		4.21155	

	Price	Price-binned	Screen-Full_HD	Screen-IPS_panel
0	978	Low	0	1
1	634	Low	1	0
2	946	Low	1	0
3	1244	Low	0	1
4	837	Low	1	0

Let's drop the two unnecessary columns that have been added into the file, 'Unnamed: 0' and 'Unnamed: 0.1'.

```
[5]: df.drop(['Unnamed: 0', 'Unnamed: 0.1'], axis=1, inplace=True)
```

0.2 Using Cross validation to improve the model

Here we divide the dataset into `x_data` and `y_data` parameters. `y_data` is the "Price" attribute, and `x_data` has all other attributes in the data set.

```
[6]: y_data = df[['Price']]
x_data = df.drop('Price',axis=1)
```

The split of the data set into training and testing subests. We reserve 10% of the data set for testing purposes.

```
[9]: x_train, x_test, y_train, y_test = train_test_split(x_data, y_data,
    ↳ test_size=0.1, random_state=1)
print("number of test samples :", x_test.shape[0])
print("number of training samples:",x_train.shape[0])
```

```
number of test samples : 24
number of training samples: 214
```

Let's create a single variable linear regression model using "CPU_frequency" parameter.

```
[11]: lm=LinearRegression()
lm.fit(x_train[["CPU_frequency"]], y_train)
print(lm.score(x_test[['CPU_frequency']], y_test))
```

```
print(lm.score(x_train[['CPU_frequency']], y_train))
```

```
-0.06599437350393811
```

```
0.14829792099817984
```

We run a 4-fold cross validation on the model and print the mean value of R^2 score along with its standard deviation.

```
[21]: Rcross = cross_val_score(lm, x_train[['CPU_frequency']], y_train, cv=4)
print("The mean of the folds are", Rcross.mean(), "and the standard_
      ↪deviation is", Rcross.std())
```

```
The mean of the folds are 0.12738818019555015 and the standard deviation is
0.08317058010912026
```

0.3 Overfitting

We are splitting the data set into training and testing components again, this time we reserve 50% of the data set for testing.

```
[47]: x_train1, x_test1, y_train1, y_test1 = train_test_split(x_data, y_data,
      ↪test_size=0.5, random_state=0)
print("number of test samples :", x_test1.shape[0])
print("number of training samples:", x_train1.shape[0])
```

```
number of test samples : 119
```

```
number of training samples: 119
```

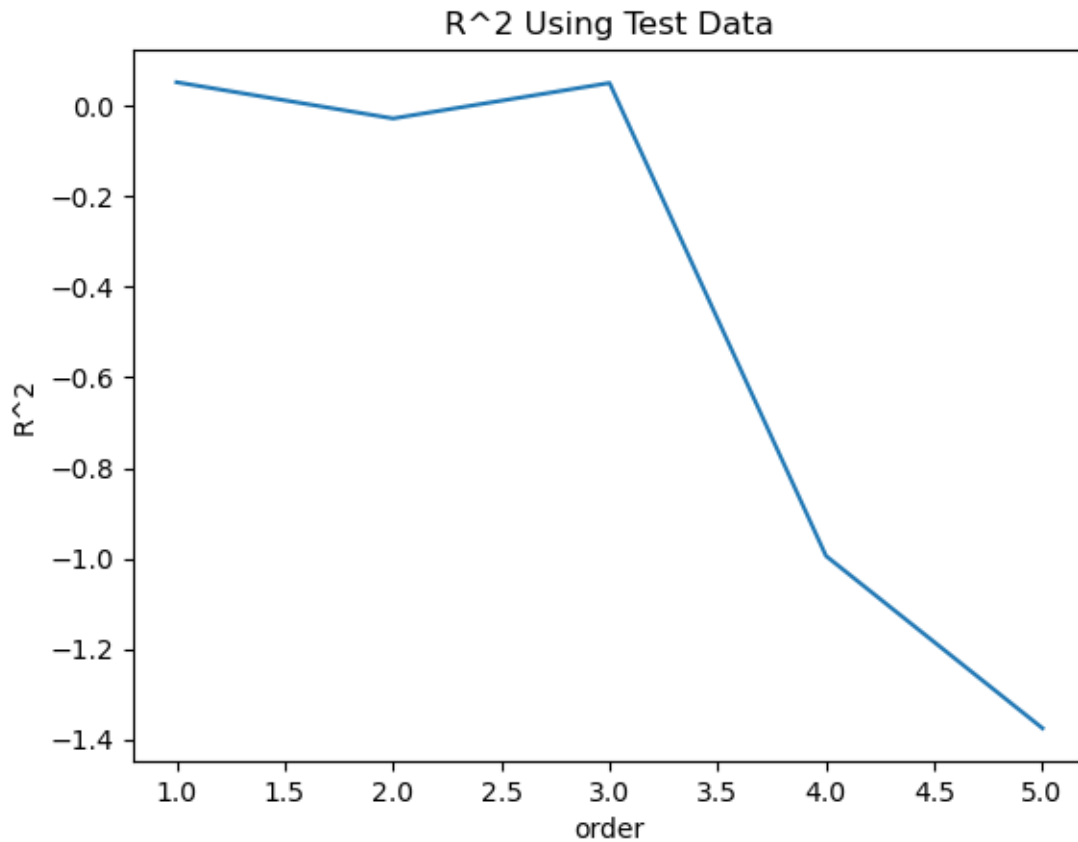
To identify the point of overfitting the model on the parameter “CPU_frequency”, we need to create polynomial features using the single attribute. We will evaluate the R^2 scores of the model created using different degrees of polynomial features ranging from 1 to 5.

```
[76]: R_test = []
order = [1, 2, 3, 4, 5 ]
for n in order:
    pr = PolynomialFeatures(degree=n)
    x_train_pr = pr.fit_transform(x_train1[["CPU_frequency"]])
    x_test_pr = pr.fit_transform(x_test1[['CPU_frequency']])
    lm.fit(x_train_pr, y_train1)
    R_test.append(lm.score(x_test_pr, y_test1))
```

Let's plot the values of R^2 scores against the order.

```
[74]: plt.plot(order, R_test)
plt.xlabel('order')
plt.ylabel('R^2')
plt.title('R^2 Using Test Data')
```

```
[74]: Text(0.5, 1.0, 'R^2 Using Test Data')
```



Note that starting from the third order, r^2 decreases

0.4 Ridge Regression

Now, let's create a polynomial feature model that uses all these parameters with degree=2.

```
[59]: pr = PolynomialFeatures(degree=2)
x_train_pr1 = pr.fit_transform(x_train1[["CPU_frequency", 'CPU_frequency', 'RAM_GB', 'Storage_GB_SSD', 'CPU_core', 'OS', 'GPU', 'Category']])
x_test_pr1 = pr.fit_transform(x_test1[["CPU_frequency", 'CPU_frequency', 'RAM_GB', 'Storage_GB_SSD', 'CPU_core', 'OS', 'GPU', 'Category']])
```

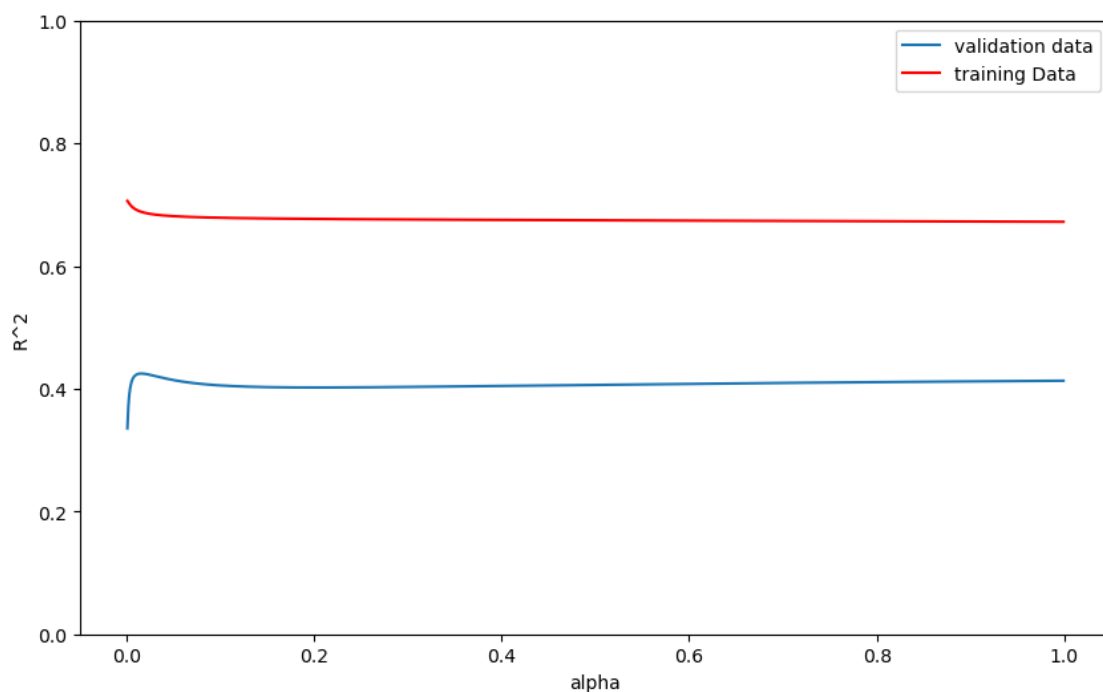
Creation of a Ridge Regression model and evaluation of the hyperparameter alpha ranging from 0.001 to 1 with increments of 0.001. .

```
[61]: R_test = []
R_train = []
Alpha = np.arange(0.001,1,0.001)
pbar = tqdm(Alpha)
for alpha in pbar:
    RigeModel = Ridge(alpha=alpha)
    RigeModel.fit(x_train_pr1, y_train1)
    test_score, train_score = RigeModel.score(x_test_pr1, y_test1),
    →RigeModel.score(x_train_pr1, y_train1)
    pbar.set_postfix({"Test Score": test_score, "Train Score":
    →train_score})
    R_test.append(test_score)
    R_train.append(train_score)
```

The plot of R^2 values for training and testing sets with respect to the value of alpha

```
[63]: plt.figure(figsize=(10, 6))
plt.plot(Alpha, R_test, label='validation data')
plt.plot(Alpha, R_train, 'r', label='training Data')
plt.xlabel('alpha')
plt.ylabel('R^2')
plt.ylim(0, 1)
plt.legend()
```

```
[63]: <matplotlib.legend.Legend at 0x7ae1dea00050>
```



0.5 Grid Search

Here we use GridSearchCV to identify the value of alpha for which the model performs best. We assume the set of alpha values to be used as

{0.0001, 0.001, 0.01, 0.1, 1, 10}

```
[65]: param = [{'alpha': [0.0001, 0.001, 0.01, 0.1, 1, 10]}]
```

```
[69]: RR = Ridge()
Grid1 = GridSearchCV(RR, param, cv=4)
Grid1.fit(x_train1[['CPU_frequency', 'RAM_GB', 'Storage_GB_SSD',
→ 'CPU_core', 'OS', 'GPU', 'Category']], y_train1)
```

```
[69]: GridSearchCV(cv=4, estimator=Ridge(),
          param_grid=[{'alpha': [0.0001, 0.001, 0.01, 0.1, 1, 10]}])
```

```
[79]: BestRR=Grid1.best_estimator_
print(BestRR.score(x_test1[['CPU_frequency', 'RAM_GB', 'Storage_GB_SSD',
→ 'CPU_core', 'OS', 'GPU', 'Category']], y_test1))
```

0.300990504869182